

Sistema de Gerenciamento de varejo

João Vitor Ribeiro e Thalita Câmara

23 de junho de 2025

1. Descrição do sistema

O sistema de gerenciamento de varejo foi desenvolvido para auxiliar empreendedores do ramo varejista a manterem um controle e fluxo de seu estoque, lista de funcionários empregados e registro de vendas. Somado a isto, a atribuição de janelas para os próprios funcionários atuarem na atualização periódica das informações do varejo, mediante sua função seguindo as regras de negócio. O Sistema possui uma interface gráfica que permite a navegação fluida e intuitiva do usuário entre as funcionalidades presentes no gerenciador, a qual se altera mediante as necessidades do usuário, sendo ele o empreendedor, ou seja, o dono do estabelecimento e para os funcionários. Além de manter rotas de comunicação entre os diferentes setores do sistema.

2. Especificação do Sistema

O programa foi desenvolvido utilizando a linguagem de programação Java, paralelamente com o software Scene builder para gerar arquivos Fxmls juntamente com esqueletos dos controles necessários para acessar os elementos gráficos presentes nos Fxmls e o Apache Maven para automação de compilação e gerenciamento de dependências. Na fase de idealização e planejamento da construção deste sistema foram discutidos e propostos diversas funcionalidades que se adequassem às necessidades reais do público alvo. A seguir está à descrição das funcionalidades implementadas:

1. Criar conta

- a. Funcionalidade que permite que novos usuários (apenas o empreendedor) realizem seu cadastro na plataforma informando dados básicos, como nome, e-mail e senha. Após o preenchimento e validação das informações, a conta é criada e o usuário passa a ter acesso às funcionalidades do sistema de forma segura.

2. Verificação de login

- a. Funcionalidade responsável por receber o nome do usuário, senha e seu

respectivo cargo (Empreendedor ou Funcionário). E então, caso esteja presente nos registros de usuários, leve-o para sua respectiva área do sistema.

3. Acesso às informações do usuário

- a. Funcionalidade encarregada de acessar as informações presentes no usuário atual, além de atribuí-las aos elementos presentes na interface gráfica.

4. Acesso ao estoque

- a. Funcionalidade responsável por acessar o estoque correspondente ao estabelecimento que o usuário trabalha ou possui. Somado à capacidade de alterar registros e enviar as modificações para todos.

5. Acesso a lista de funcionários

- a. Funcionalidade exclusiva do dono do estabelecimento, a qual permite a visualização, adição, edição e remoção de sua lista de funcionários contratados.

6. Registro de Vendas

- a. Funcionalidade exclusiva da classe Atendente para registrar as vendas durante seus turnos de trabalho.

7. Registro de Gasolina

- a. Funcionalidade criada para manter o acompanhamento da quantidade de gasolina gasta pelo entregador.

Como trabalho futuro, pretende-se implementar um banco de dados para criar memória persistente do sistema, criar a funcionalidade Tarefas() responsável por atribuir atividades aos funcionários e atualizar o gerenciador para aumentar sua extensão de conexão com as outras classes.

3. Conteúdos usados no projeto

A seguir será detalhado como este projeto engloba os conteúdos ministrados na matéria curricular, programação orientada a objetos, por meio de observações objetivas e exemplificações com imagens do código-fonte.

3.1 Classe, objetos, métodos assessores, toString

Este trabalho tem como base a criação de classes representando entidades reais, com seus respectivos métodos para acessar seus atributos ou para realizar ações exclusivas, somado ao método toString. As principais classes criadas foram: Empreendedor, Funcionário, Gerenciador, Estoquista, Atendente e Entregador. Segue o exemplo abaixo:

```

//atributos
private String nomeEmpreendimento;
private int id_empresa;
private double vendas;
private double compras;
private ArrayList<Funcionario> listaFuncionarios;
//Construtor
public Empreendedor(String nome, String senha, String email,String nomeEmpreendimento) {
    super(nome, senha, email);
    this.nomeEmpreendimento = nomeEmpreendimento;
    id_empresa = ThreadLocalRandom.current().nextInt(0, 999);
    //TODO Auto-generated constructor stub
}
//getters and setters
public String getNomeEmpreendimento() {
    return nomeEmpreendimento;
}

public void setNomeEmpreendimento(String nomeEmpreendimento) {
    this.nomeEmpreendimento = nomeEmpreendimento;
}

@Override
public String toString() {
    return "Empreendedor [nomeEmpreendimento=" + nomeEmpreendimento + ", id_empresa=" + id_empresa +
        + vendas + ", compras=" + compras + ", listaFuncionarios=" + listaFuncionarios + "]\n";
}

```

Figura 1. Exemplo de Classe.

3.2 Encapsulamento

Desde o início do projeto, o pilar do encapsulamento se fez presente para tomada de decisões acerca de como determinados atributos e até métodos deveriam ser acessados. Durante o desenvolvimento do sistema foram tomadas diferentes decisões acerca do tipo de modificador de acesso que deveria ser aplicada. Por exemplo: atributos que deveriam ser compartilhados com outras classes foram atribuída o modificador `protected` para serem acessadas pelas outras classes pertencentes do mesmo pacote, porém majoritariamente os atributos são privados e acessados por meio de métodos assessores. Observe um exemplo abaixo:

```

public class AtendenteController implements Initializable {
    protected static Atendente funcSel = new Atendente(nome: null, senha: null, email: null,
    private Produtos produtoSel;
    private float valorProdTot = 0;
    private int quantProdTot = 0;

    public AtendenteController(){
        funcSel = (Atendente) Gerenciador.getSelecioneado();
    }

    @FXML
    private Text FuncFuncao;

    @FXML
    public void MinusItem(ActionEvent event) {

        valorProdTot -= Integer.parseInt(quantArea.getText())*produtoSel.getValor();
        quantProdTot -= Integer.parseInt(quantArea.getText());
        TotalText.setText("Total: " + String.valueOf(valorProdTot));
        QuantTotal.setText("Quantidade total: " + String.valueOf(quantProdTot));
    }
}

```

Figura 2. Exemplo de Encapsulamento.

3.3 Uso de construtores e Objetos;

O programa exigiu a criação de diversas classes, portanto cada classe recebeu seu respectivo Construtor para serem futuramente instanciados em objetos. Ao longo da construção do sistema foi observado os diferentes tipos de construtores, sendo eles o vazio, com todos os atributos preenchidos com valores nulos e aqueles que se fez necessário atribuir um valor para cada um dos atributos manualmente. Abaixo segue dois exemplos de um Construtor da classe Empreendedor:

```

//Construtor
public Empreendedor(String nome, String senha, String email,String nomeEmpreendimento) {
    super(nome, senha, email);
    this.nomeEmpreendimento = nomeEmpreendimento;
    id_empresa = ThreadLocalRandom.current().nextInt(0, 999);
    //TODO Auto-generated constructor stub
}

```

Figura 3. Exemplo de uso de Construtores.

```

adicionar(new Empreendedor(nome: "Thalita", senha: "123", email: "nome@gmail.com", nomeEmpreendimento: "Designer Com Thalita"));
adicionar(new Empreendedor(nome: "joão", senha: "12", email: "jeve@gmail.com", nomeEmpreendimento: "hq"));
adicionar(new Funcionario(nome: "sofia", senha: "1234", email: "email", salario: 0, cargo: "Estoquista", data_de_admi... null, "Thal");
adicionar(new Entregador(nome: "gabriel", senha: "02", email: "email", salario: 2500, cargo: "Entregador", LocalDate.of(1908, 01,
adicionar(new Atendente(nome: "maria", senha: "111", email: "email", salario: 1200, cargo: "Atendente", LocalDate.of(2000, 02, 01),

```

Figura 4. Exemplo de objetos.

3.4 Composição;

A composição nesse sistema foi utilizada uma única vez para atribuir a classe Carrinho para o Atendente que já se encontrava como classe filha de Usuário. Sendo que se mostrou muito eficaz para futuras atualizações em que o atendente poderá atribuir uma lista inteira de produtos para a compra. Segue o exemplo:

```

public class Atendente extends Funcionario{
    private Carrinho<Produtos> carinho;

    public Atendente(String nome, String senha, String email, float sal
        super(nome, senha, email, salario, cargo, data_de_admissao, che
        this.carinho = new Carrinho<>();
    }

    public Carrinho<Produtos> getCarinho() {
        return carinho;
    }

    public void setCarinho(Carrinho<Produtos> carinho) {
        this.carinho = carinho;
    }
}

```

Figura 5. Exemplo de Composição.

3.5 Herança;

A herança foi o principal elemento que permitiu a reutilização de código entre as classes. A qual, foi criada uma superClasse chamada Usuário e suas duas principais subclasses: Empreendedor e Funcionário, sendo este último a raiz de 3 subclasses: Atendente, Estoquista e Entregador. Além do reuso de código, também foi possível implementar novos atributos e métodos sem a colisão de lógica. Observe o exemplo de Empreendedor (subclasse) herdando de Usuário(superClasse):

```

public class Empreendedor extends Usuario{

    //atributos
    private String nomeEmpreendimento;
    private int id_empresa;
    private double vendas;
    private double compras;
    private ArrayList<Funcionario> listaFuncionario
    //Construtor
    public Empreendedor(String nome, String senha,
        super(nome, senha, email);

```

Figura 6. Exemplo de herança.

3.6 Coleções (ArrayList)

Na ausência de um banco de dados conectado ao sistema, as coleções se apresentaram como uma alternativa rápida e eficiente para solucionar o problema da memória não persistente, a qual foi utilizada para armazenar a lista de usuários, sendo chamada sempre que necessário. Os arraylists também foram implementadas como um atributo na classe Funcionario e suas filhas com o papel de armazenar tarefas atribuídas pelo Empreendedor. Segue o exemplo do uso de coleção:

```

public class Gerenciador {
    // variaveis static(acessíveis a todos)
    private static ArrayList<Usuario> listaUsuario = new ArrayList<Usuario>()

    public static ObservableList<Funcionario> listaFuncionarios = FXCollections.observableArrayList()
    private static ObservableList<Produtos> listaEstoque = FXCollections.observableArrayList()
    private static Usuario user;

```

Figura 7. Exemplo de coleção.

3.7 Classes e métodos abstratos e Interface

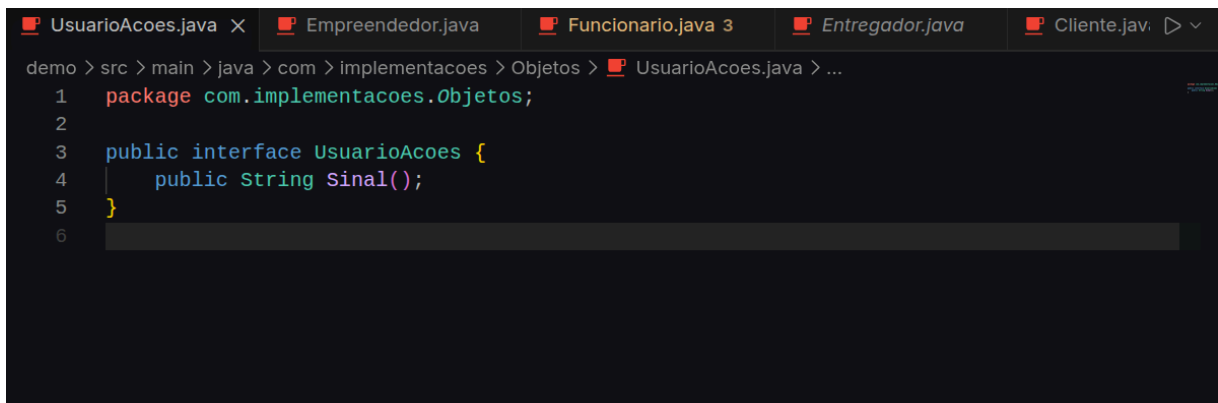
A superClasse Usuário foi definida como abstrata, pois desde o planejamento foi observado que esta classe não seria instanciada e seus métodos seriam divergentes de acordo com suas diferentes ramificações, portanto tanto a classe quanto os seus métodos, exceto os assessores, são abstratos. Segue o exemplo:

```

public abstract class Usuario {
    //atributos privados
    private String nome;
    private String senha;
    private String email;
    //Construtor -- os parametros são obrigatorios, afinal
    public Usuario(String nome, String senha, String email) {
        this.nome = nome;
        this.senha = senha;
        this.email = email;
    }
    public abstract String Sinal();
}

```

Figura 8. Exemplo de classe e métodos abstratos.



```

demo > src > main > java > com > implementacoes > Objetos > UsuarioAcoes.java > ...
1  package com.implementacoes.Objetos;
2
3  public interface UsuarioAcoes {
4      public String Sinal();
5  }
6

```

Figura 9 . Exemplo de Interface.

3.8 Polimorfismo de inclusão/sobreposição e Sobrecarga

O uso de métodos vindos de uma superClasse foram recorrentes, portanto o uso do polimorfismo de inclusão foi essencial para diferentes situações, como por exemplo o método Sinal() proveniente da superClasse Usuário, a qual foi sobrescrita em cada classe filha para atender as necessidades específicas de cada classe. Enquanto a sobrecarga foi utilizada para diversificar o tipo de entrada que um mesmo método poderia receber, a qual exemplificando foi utilizada para criar o método de gasto de gasolina, a qual pode receber a quantidade gasta ou apenas remover uma quantidade de gasolina padrão, sendo este método da classe Entregador. Segue os exemplos abaixo:

```

@Override
public String Sinal() {
    // TODO Auto-generated method stub
    return ("Funcionario operando");
}

```

Figura 10. Exemplo Polimorfismo de inclusão

```

public void setGasolina_gasta(int gasolina_gasta) {
    this.gasolina_gasta = gasolina_gasta;
}

public void setGasolina_gasta() {
    this.gasolina_gasta = 5;
}

```

Figura 11. Polimorfismo de sobrecarga

3.9 Downcasting e Upcasting

Para evitar a criação de diversas coleções para armazenar os diferentes tipos de usuário, foi criado um arraylist que armazena se qualquer objeto que possuisse como superClasse a classe Usuário, dessa maneira o sistema realiza o Upcasting de forma implícita. Porém, após selecionado o usuário se fez necessário realizar o Downcasting para ter acesso aos atributos e métodos da subclasse, imprescindíveis para a execução do programa. Abaixo segue dois exemplos de Upcasting e Downcasting:

```

//Métodos
public static void adicionar(Usuario dono) {
    listaUsuario.add(dono); // adicionar um usuario a um
}

static {
    adicionar(new Empreendedor(nome: "Thalita", senha: "123456789"));
    adicionar(new Empreendedor(nome: "joão", senha: "12", senha: "123456789"));
    adicionar(new Funcionario(nome: "sofia", senha: "123456789"));
    adicionar(new Entregador(nome: "gabriel", senha: "02", senha: "123456789"));
    adicionar(new Atendente(nome: "maria", senha: "111", senha: "123456789"));
}

```

Figura 12. Exemplo Upcasting


```
//Outro que recebe do Gerenciador
public DonoController() {
    if (Gerenciador.getSelecionado() instanceof Empreendedor) {
        dono = (Empreendedor) Gerenciador.getSelecionado();
    }
}
```

Figura 13. Exemplo Downcasting

3.10 Exceções

Como se trata de um sistema que será utilizado por usuários reais, se fez necessário a criação de tratamento de erros e exceções tanto para lidar com os erros humanos, quanto para tratar exceções durante a execução do sistema. A qual o mais utilizado foi o `IOException` para lidar com uma rota errada dos arquivos Fxmls. Segue o exemplo:

```
//inicializar a janela e o palco
public void start() {
    //Adicionamos um try_catch ou seja tratamento de erros e exções
    try {
        //1. carrega o fxmll
        java.net.URL fxmUrl = com.implementacoes.App.class.getResource("/cena.fxml");
        FXMLLoader loader = new FXMLLoader((fxmUrl)); //Criando o FXMLOADER
        Parent root = loader.load(); //Carregando a cena dentro do root

        //Criar a cena
        Scene cenaDemitirEditar = new Scene(root); //atribuimos o root á cena

        //Criando a janela
        Stage JanelaDemitirEditar = new Stage();
        JanelaDemitirEditar.setScene(cenaDemitirEditar); //atribuindo a cena á janela
        JanelaDemitirEditar.setTitle(value: "Demitir e Editar"); //atribuindo o título

        //mostrando
        JanelaDemitirEditar.show(); //hora do show..ksskks

    } catch (IOException ex) {
        ex.printStackTrace();
        System.out.println("Erro ao carregar o arquivo FXML. Verifique o caminho do arquivo.");
    }
}
```

Figura 14. Exemplo Exceções

3.11 Complexidade

A complexidade do projeto não se agregar apenas no uso de uma biblioteca de interface gráfica em conjunto com um software externo, mas sim na criação de regras, rotas e conhecimentos aprofundados da própria linguagem para estruturar um sistema de Gerenciamento que se comunica em tempo real com as outras classes, além da atualização constante da interface para que o usuário possa visualizar as consequências de suas ações. Ainda acerca do usuário, Soma se os possíveis erros humanos que podem ocorrer durante a execução do programa, as quais tiveram que ser testadas e tratadas. Segue o exemplo:

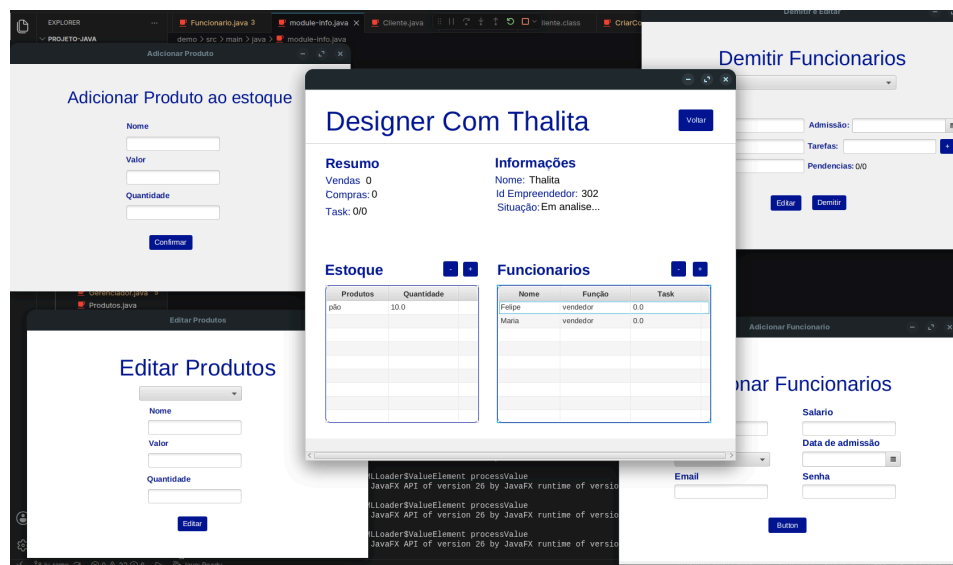


Figura 15. Exemplo de entrada e saída de dados do sistema

Importante acentuar as diversas dificuldades ao longo do projeto, sendo algumas delas à preparação do ambiente para utilização de uma biblioteca externa ao Java, inicializar sempre que ativada o programa uma série de atributos de diferentes classes, a comunicação entre as diferentes classes sem causar a conflito ou perda das informações anteriormente presentes e o uso de composição no sistema.

4. Conclusão

Conclui-se que o sistema de gerenciamento de loja foi desenvolvido com o propósito de atender às necessidades de pequenos empreendimentos, disponibilizando uma ferramenta capaz de auxiliar no acompanhamento e na gestão da evolução do negócio por meio de registros digitais realizados de forma intuitiva, prática e eficiente. Além de otimizar a organização das informações, o sistema foi concebido com uma arquitetura que favorece sua manutenção e evolução, permitindo a implementação de novas funcionalidades e adaptações para atender às demandas que possam surgir futuramente.